DFlash 2: Keep Drafting Parallel

AUGUST 18, 2026

Inference is the bottleneck of the agent era. Agents read, plan, and call tools, often for hours or days. They consume tokens at a rate chat never approached. Every one of those tokens takes a full forward pass over the model. At Inco AI, we are building the inference stack scaled to the token economics of tomorrow. This post is a sneak peek.

Our team released DFlash in January; it now runs in SGLang, vLLM, TensorRT-LLM, and llama.cpp. NVIDIA measured up to 15× throughput with it on Blackwell GPUs; Google reported 3× more tokens per second on TPUs; CoreWeave's production Kimi K2.7 Code endpoint, the fastest for that model on Artificial Analysis, runs DFlash by default. The ecosystem now builds on it: NVIDIA, Red Hat, and Modal have all published DFlash drafters; Meta (Muse Glimmer), Poolside (Laguna), Xiaomi (MiMo-V2.5-Pro), and NVIDIA (Nemotron 3.5 Lightning) ship official drafters with their own models. On Hugging Face, DFlash models have been downloaded **more than 3.5 million times** (as of August 2026).

Speculative decoding is a core piece of the modern inference stack.[1] A small draft model guesses a block of tokens, and the target model verifies the whole block in one forward pass. Good guesses turn one pass into several tokens; bad ones just get thrown away. For years, though, the draft itself stayed ***autoregressive***: one token at a time. DFlash made it one-pass too: the entire block, every position, predicted ***in parallel***.

DFlash 2 drafting for Qwen3.8-27B on an Apple M5 Max with oMLX, side by side with autoregressive decoding.

DFlash 2 pushes parallel drafting one step further: **over 20% more output from every verification pass, for around 1% added cycle latency**, with the output provably unchanged. Across benchmarks the gain runs 16–25%. With the Qwen3.8-27B drafter released today, SGLang serves at **2.7–3.4× the throughput of autoregressive decoding** at batch size 1. Predicting every position independently leaves headroom in two places: choosing the right tokens and holding accuracy to the end of the block. DFlash 2 recovers both without giving up the one-pass design.

Run It Now

DFlash 2 already runs in the mainstream inference engines:

SGLang vLLM llama.cpp Ollama oMLX

```
pip install "sglang[all] @ git+https://github.com/sgl-project/sglang.git#subdir

python -m sglang.launch_server \
  --model-path Qwen/Qwen3.8-27B \
  --speculative-algorithm DFLASH \
  --speculative-draft-model-path incoai/Qwen3.8-27B-DFlash2 \
  --speculative-num-draft-tokens 8
```

The Right Tokens Are Already There

DFlash predicts every position independently, in parallel. Each pick is plausible on its own. Yet nothing makes them fit together, and an incoherent block is cut short at verification. Recent methods such as Domino and DSpark buy coherence with sequential heads that rewrite each position's full-vocabulary distribution. But is that costly autoregressive correction really necessary?

No. The evidence is already in DFlash's own candidate lists. Take the first position: DFlash's top pick is right 85.4% of the time, but the right token is in its top 16 candidates 99.5% of the time. Even when the top pick is wrong, the right token is usually on the list.

METRIC	0	1	2	3	4	5	6	ACCEPTANCE LENGTH
Recall@1	85.4%	80.3%	79.4%	78.3%	77.5%	75.9%	72.9%	4.2
Recall@16	**99.5%**	**97.3%**	**94.8%**	**92.6%**	**90.8%**	**89.4%**	**87.8%**	**6.7**

Table 1. Recall@1 (how often the top pick is right) and Recall@16 (how often the right token is in the top 16) at each draft position, conditioned on every earlier position being right. Five-layer Qwen3-4B DFlash on GSM8K. Acceptance length includes the verifier's next token.

An oracle that always picks the right candidate from the top 16 would lift the acceptance length from 4.27 to 6.79. **That gap is pure selection headroom.** We just need to select the right path through the candidates.

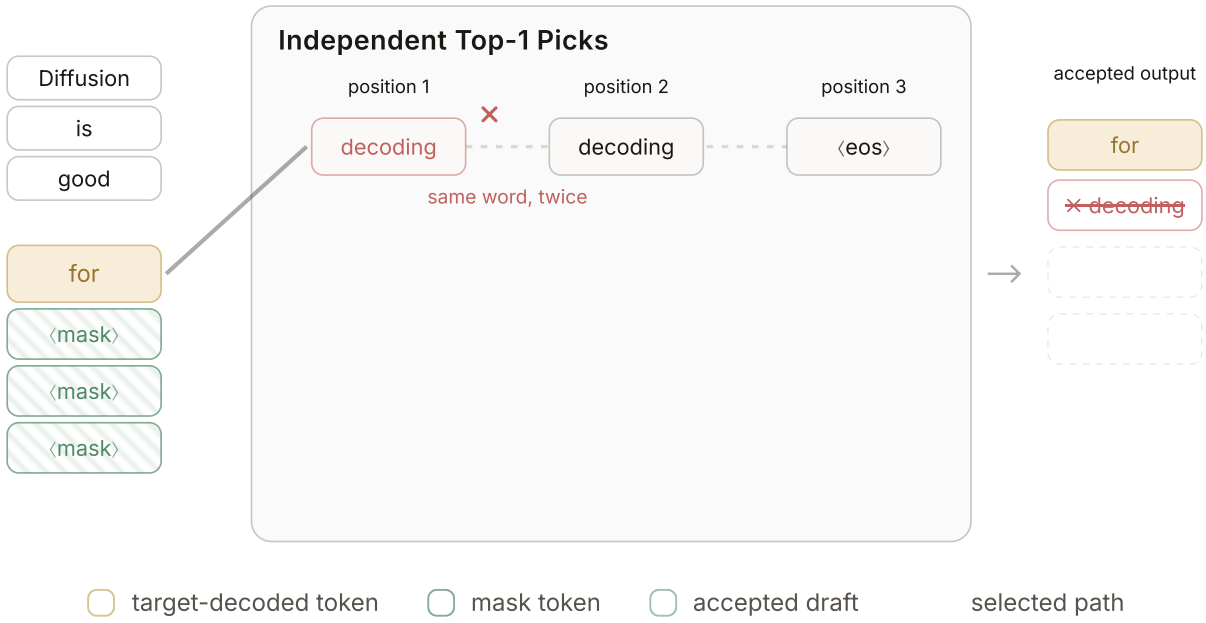


Figure 1. The selector in one cycle. With DFlash alone, each position keeps its top pick; here two neighbors both pick the same word, and the stutter dies at verification. DFlash 2 keeps each position's top candidates, and the selector traces one coherent path through them; here, the whole block survives.

A Lightweight Path Selector

Coherence is mostly local: a candidate's fit depends mainly on the token just before it, so scoring neighboring pairs should be enough. DFlash 2 keeps the top 16 candidates at each position and scores every adjacent pair: for predecessor a and current candidate b,

$$S_t(a, b) = U_t(b) + \langle A(a) \odot H(h_t), B(b) \rangle.$$

The score has two parts. The first, $U_t(b)$, is DFlash's own logit: how much the drafter already liked b on its own. The second asks how well b follows a: A and B give each token a compact 256-dimensional embedding, and the two embeddings are matched under a context gate $H(h_t)$ that decides which parts of the match count. In essence, this is a low-rank bilinear attention over adjacent candidates.

Scoring stays fully parallel. Every adjacent pair at every position is scored in one shot, with no extra backbone or LM-head pass. The only sequential work is the final walk over precomputed scores: starting from the last verified token, greedy follows the best successor at each step, sampling draws from the same scores, and rejection sampling restores the exact target distribution.

METHOD	PARAMS	LATENCY	T = 0	T = 1
DFlash	—	—	4.27	3.78
+ DSpark correction	+77.8M	+9.6%	4.49	4.08
+ path selection (ours)	**+2.0M**	**+0.6%**	**4.61**	**4.25**

Table 2. Acceptance length with path selection alone (no convolution), for five-layer Qwen3-4B on GSM8K. Overheads are relative to plain DFlash: parameters added to the drafter, and added draft–verify cycle latency.

The selector improves DFlash by **0.34** tokens at $T = 0$ and **0.47** at $T = 1$. It beats the DSpark correction in both settings with roughly 40× fewer parameters and 16× lower latency overhead. Choosing is cheaper than predicting. And there is still

room: the oracle reaches 6.79. Pairwise scoring is the simplest selector we could think of, and we believe there is plenty to explore.

Suffix Decay Is a Local Problem

We also noticed both recall rows above decline toward the end of the block. Even the oracle decays: with perfect selection, accuracy still falls from 99.5% at the first position to 87.8% by the last. No selector can fix that, because the candidates themselves are running out. We call this **suffix decay**, and it is a backbone problem.

One suspect is capacity: a five-layer backbone may be too small to preserve dependencies across the block. If that is right, depth should help most at later positions. And it does! 3-, 5-, and 15-layer DFlash models are almost identical at the first position, and fan apart down the block. But depth is indiscriminate: ten extra attention blocks add capacity everywhere, even at the early positions that had little left to gain, and erase much of the efficiency that makes DFlash attractive.

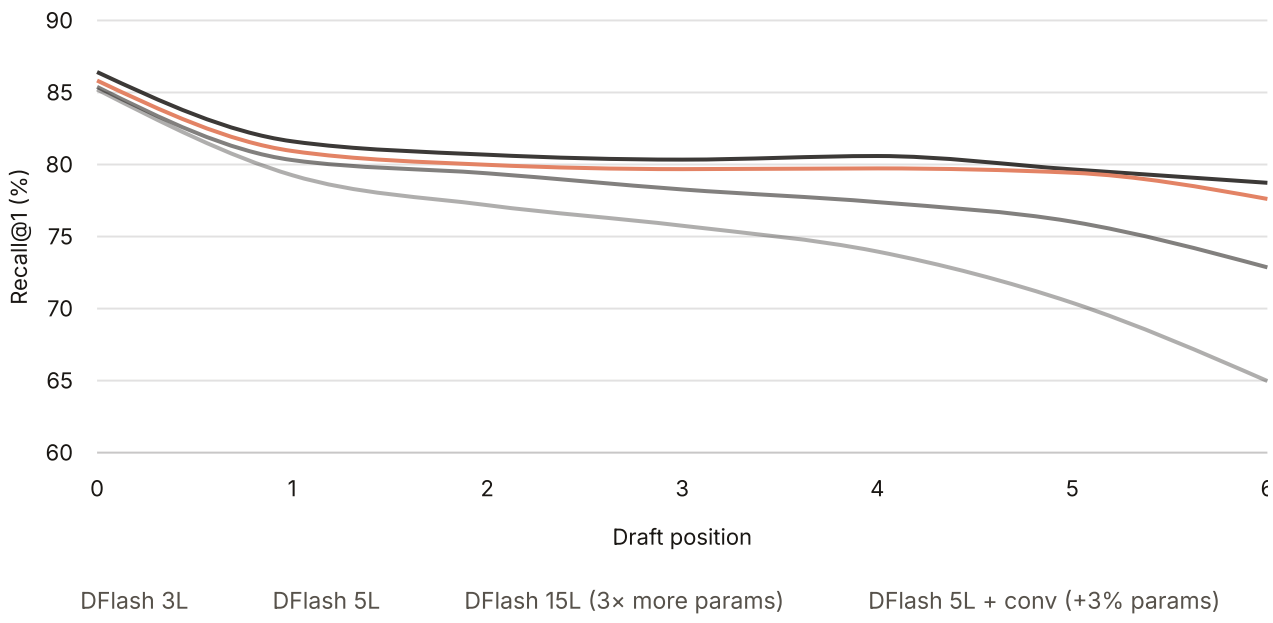


Figure 2. Qwen3-4B Recall@1 on GSM8K at T=0, conditioned on every earlier position being right. All drafters are trained under the same setup; the convolutional model is evaluated without the selector. Its convolutions add 3% parameters and 0.7% cycle latency; the ten extra layers of 15L add 15.2%.

We want a targeted fix, and DFlash's attention shows where. It has two jobs: read the context before the block, and model the dependencies inside. But it spends less and less on the second: the block's share of attention falls from **30% in Layer**

1 to 8% in Layer 5, and what remains concentrates in a shrinking handful of heads. So we split the jobs: a dedicated module takes the within-block work, and attention keeps reading the context.

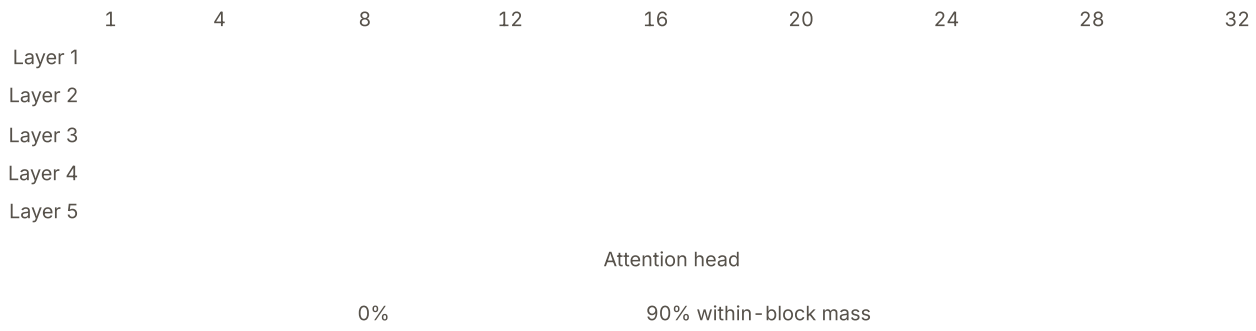


Figure 3. Within-block attention by head in five-layer Qwen3-4B DFlash. Brighter cells mark heads that spend more attention on the draft block; in later layers the within-block mass shrinks and concentrates in a few heads.

A Lightweight Local Convolution

The within-block work is short-range to begin with: a block spans only 4 to 16 tokens, and the tightest dependencies sit between neighbors. The natural operator is a short convolution: two taps, one on the current position and one reaching one position back, with weights that adapt to the content. Following Canon Layers, Dynamic Short Convolutions, and Convolution for Large Language Models, we insert this two-tap dynamic depthwise convolution before and after each attention and feed-forward sublayer:

$$\mathrm{Conv}_k(x)_t = k_{t,0} \odot x_t + k_{t,1} \odot x_{t-1}.$$

Each coefficient combines a learned base kernel with a small correction computed from the current hidden state; every 16 channels share one correction. The first position reads the last verified token's representation, and every later position reads its predecessor's. Information crosses the block while all positions still compute in parallel.

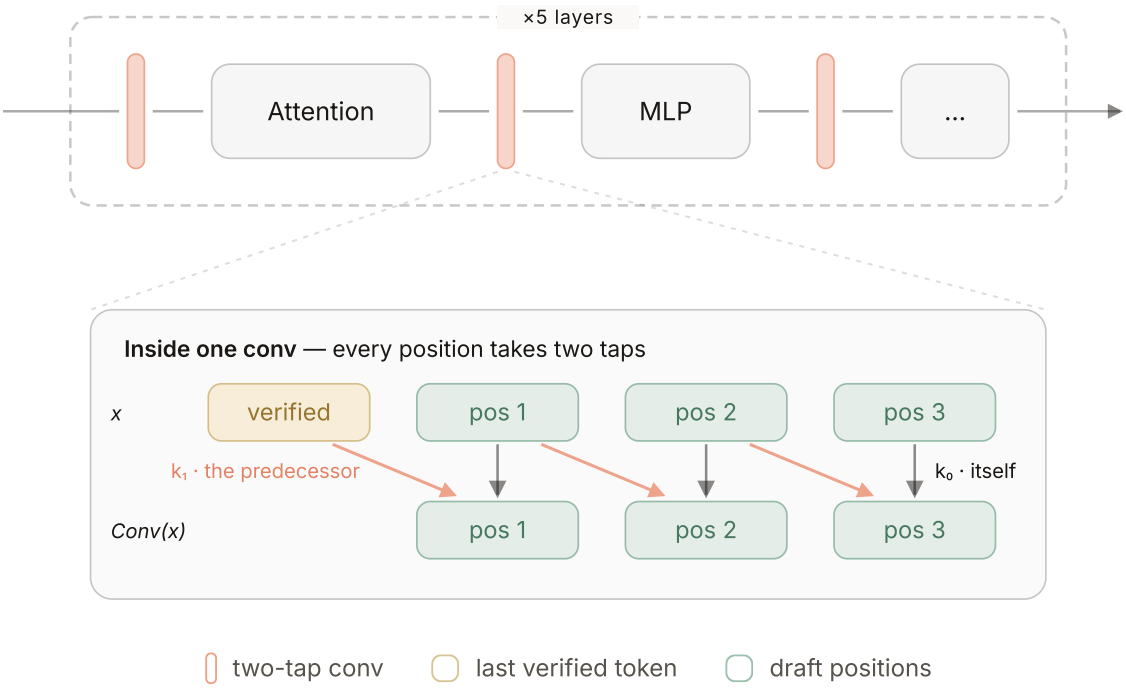


Figure 4. The two-tap dynamic convolution. One sits before and after each attention and MLP sublayer of every drafter layer. Inside it, each position mixes its own representation with its predecessor's, and the first position reads the last verified token.

The convolution is block-local and stateless, so it drops into DFlash without changing attention, the LM head, or verification.

With only **16.5M added parameters (3%)**, five-layer DFlash with convolution comes close to 15-layer DFlash, substantially reducing suffix decay. The convolutions add **0.7%** to draft–verify cycle latency; ten more Transformer layers add 15.2%. Average within-block attention across Layers 4 and 5 also falls from **9.4% to 0.5%**, consistent with the convolution absorbing the local work while attention goes back to reading the context. A kernel reaching one position back recovers most of what ten extra layers buy: suffix decay is mostly a *local* problem.

Putting It Together

So far, the selector and the convolution have been measured separately; the full comparison below puts them together. We trained the DFlash and DSpark drafters ourselves under matched setups, while MTP ships with the model.

Qwen3.5-4B

DATASET	MTP	DFLASH	DSPARK	DFLASH 2
GSM8K	4.78	4.99	5.69	**6.20**
MATH-500	5.04	5.42	6.20	**6.76**
HumanEval	4.84	5.43	5.80	**6.28**
MBPP	4.16	4.49	4.96	**5.41**
MT-Bench	3.90	4.26	4.77	**5.20**
Mean	4.54	4.92	5.49	**5.97**

Table 3. Qwen3.5-4B per-request mean acceptance length. Sampling: thinking enabled, temperature 1.0, top-p 0.95, top-k 20, presence penalty 1.5, with lossless rejection sampling.

DFlash 2 leads on every benchmark. Averaged across them, it gains **1.05 tokens over DFlash (21%)** and **0.48 over DSpark**. The upgrade stays cheap: the selector and the convolution together add only **1.3%** to the five-layer DFlash draft–verify cycle latency.

On MATH-500, the gain is visible position by position: DFlash 2 holds steady near 86% to the last position, and every baseline ends the block 6 to 9 points below it.

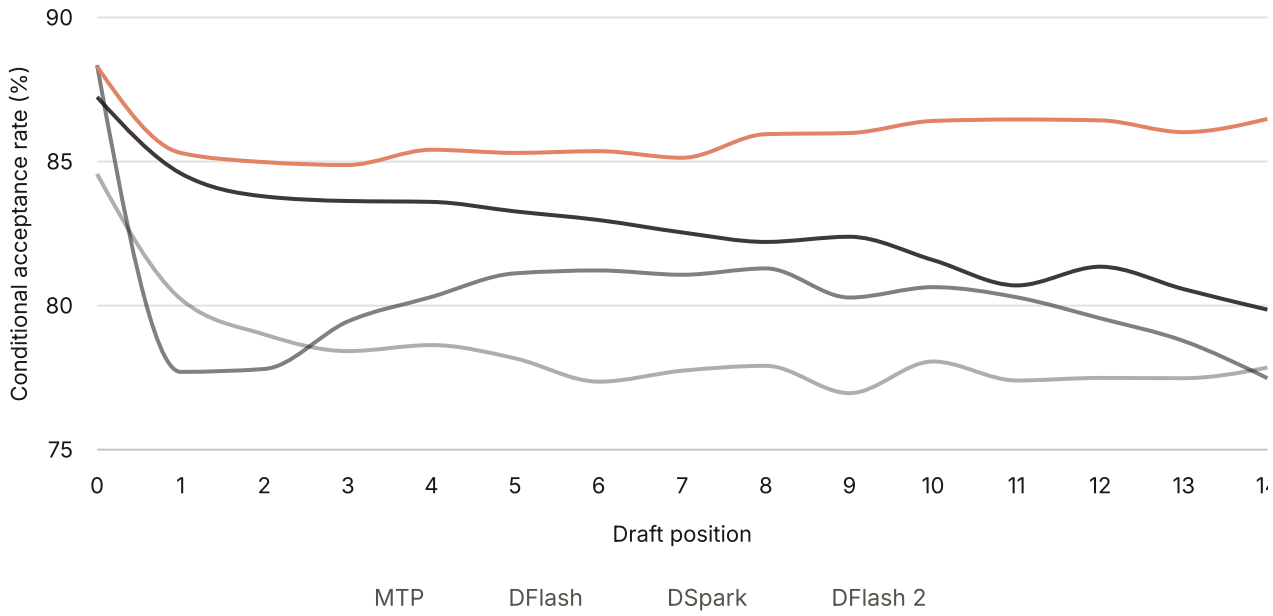


Figure 5. Qwen3.5-4B conditional acceptance rate on MATH-500, same sampling as above.

Two Drafters, Out Today

We are releasing two DFlash 2 drafters today: one for Qwen3.8-27B and one for Meta's Muse Glimmer. For Qwen3.8-27B, we compare against the model's native MTP path and a community DSpark drafter.

Qwen3.8-27B

DATASET	MTP	DSPARK	DFLASH 2
GSM8K	5.02	4.36	**5.46**
MATH-500	4.72	3.92	**5.28**
HumanEval	3.91	3.30	**4.39**
MBPP	3.99	3.51	**4.79**
MT-Bench	3.74	3.01	**4.10**
Mean	4.28	3.62	**4.80**

Table 4. Qwen3.8-27B per-request mean acceptance length with the model's default sampling and a block size of 8, against its native MTP path and a community DSpark drafter.

For Meta's Muse Glimmer, we compare against the official DFlash drafter shipped with the model and a community DSpark drafter.

Muse Glimmer

DATASET	DFLASH	DSPARK	DFLASH 2
GSM8K	5.43	5.45	**6.57**
MATH-500	5.39	5.01	**6.56**
HumanEval	4.11	4.33	**5.66**
MBPP	3.74	4.02	**5.30**
MT-Bench	3.52	3.59	**4.42**
Mean	4.44	4.48	**5.70**

Table 5. Muse Glimmer per-request mean acceptance length with the model's default sampling and a block size of 16. DFlash is the official drafter Meta ships with the model; DSpark is a community drafter.

The margins are wide: on both models, DFlash 2 averages more than **a full token** ahead of DSpark. It also beats each model's official drafter, MTP on Qwen3.8-27B and DFlash on Muse Glimmer. That translates into **2.7–3.4×** the throughput of autoregressive decoding on Qwen3.8-27B, and **3.1–4.6×** on Muse Glimmer. The model cards break the speedups down by task and concurrency.

The Bottom Line

An agent writes in an afternoon what a chatbot writes in a month, and decoding sits under every one of those tokens. DFlash 2 decodes at **close to 3× the speed of autoregressive decoding, about a third of the compute per token**, with the same output.

In seven months, DFlash went from our paper to an industry standard, with more than 3.5 million downloads. Inside the same design, DFlash 2 decodes one more full token per pass, for free. That is only one component of the serving stack. Inference is nowhere near its floor.

At Inco AI, we are building an end-to-end serving stack to keep pushing that floor lower. DFlash 2 is the first piece. Two drafters are out today on Hugging Face.

If you serve agents at scale and want to evaluate DFlash 2 in your stack, or want a drafter for a model you run, including your own fine-tunes, write to us: contact@inco.ai.

We are also hiring. If you want to help build this stack, reach out to us.

Connect the candidates. Keep drafting parallel.

Citation

Please cite this post as:

```
@misc{inco2026dflash2,
  title  = {{DFlash 2: Keep Drafting Parallel}},
  author = {{Inco AI}},
  year   = {2026},
  month  = {August},
```

```
  url     = {https://inco.ai/blog/dflash2/}
}
```

1. Modal's "Speculation Is All You Need" points out that speculative decoding is the optimization that matters for low-latency serving. We are huge fans of their work and appreciate their support and discussions since DFlash's release. ↩

DECODE NEWSLETTER

Decode with us.

Stay in the loop about research, releases, and ideas about the frontier of efficient inference

you@example.com Subscribe

One email when we publish something new. Unsubscribe anytime.

---

contact@inco.ai